

Categorical and Mixed-type Data

Chapter 4

Ashim Khadka

Data Science and Analytics

Outline

- 1 Introduction: Categorical and Mixed-type Data
- 2 Classification: Decision Trees and CART
 - Iterative Dichotomiser 3 (ID3)
 - Classification and Regression Trees (CART)
- 3 Regression: Logistic Regression
- 4 Variable Importance
- 5 Clustering

Data Types Recap

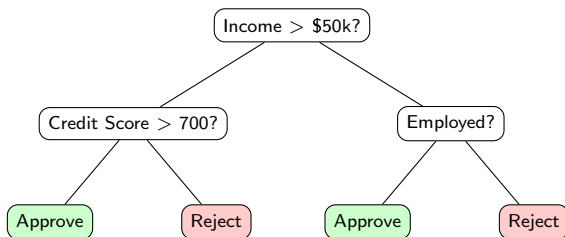
- **Continuous (numeric):** height, income, temperature.
- **Categorical (nominal):** gender, blood group, product category.
- **Ordinal:** education level, satisfaction rating (Low < Medium < High).
- **Mixed-type:** a dataset containing a combination of the above (e.g., patient records with age, blood type, diagnosis code).

Classical multivariate methods struggle

- Techniques: Euclidean distance, PCA, or ordinary least-squares regression
 - assume numeric, continuous variables
- Categorical variables violate the algebraic assumptions
 - no meaningful mean, no natural ordering
 - motivates specialized ML methods: **trees**, **logistic regression**, and **clustering**

Decision Tree

- supervised learning algorithm, use to solve
 - Regression and Classification problems
 - CARTs (Classification And Regression Tree)
- Algorithm that takes as
 - input a vector of attribute values
 - returns a “decision ” a single output value
- A decision tree **reaches** to its decision by performing a sequence of tests
 - by **divide and conquer**
- uses a tree-like model of decisions
- imitate *human decision making*

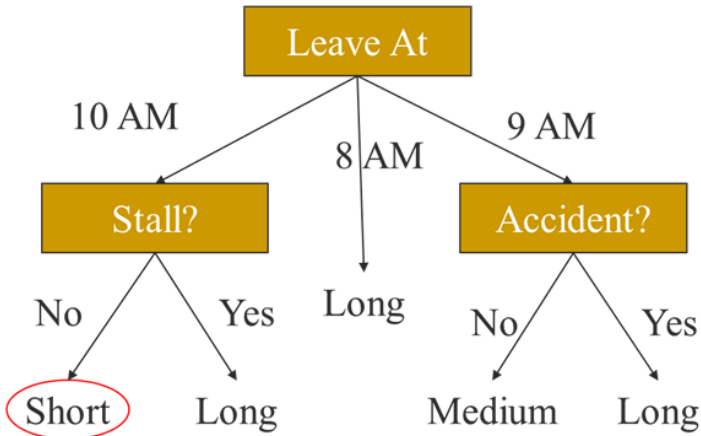


Divide and Conquer Strategy in Decision Trees

- ➊ **Divide:** dataset is split into subsets based on a feature
 - Select the best feature and split point using a criterion: **Information Gain, Gini Index**
- ➋ **Conquer:** Each subset is treated as a new problem
 - process is repeated recursively to create further splits until a stopping criterion is met (e.g., maximum depth, minimum samples per node, or pure nodes)
- ➌ **Combine:** solutions from the subproblems (leaf nodes) are combined to form the final decision tree
- ➍ each leaf provides a prediction (e.g., majority class for classification or average value for regression)

Predicting Commute Time

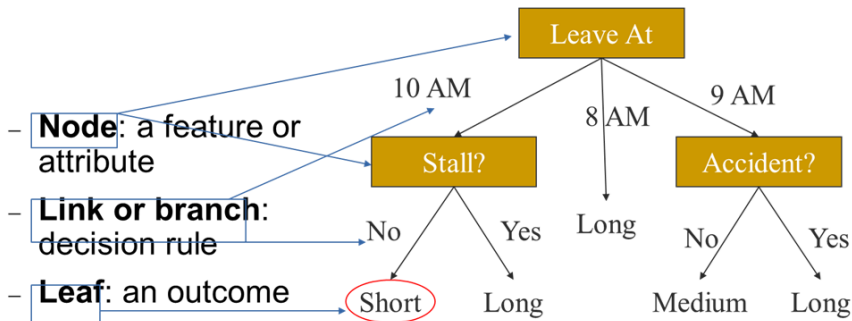
If we leave at 10 AM and there are no cars stalled on the road, what will our commute time be?



Predicting Commute Time

- Made a series of Boolean decisions and followed the corresponding branch
 - Did we leave at 10 AM?
 - Did a car stall on the road?
 - Is there an accident on the road?
- By answering each of these yes/no questions, we then concluded how long our commute might take

Components of a Decision Tree



Components of a Decision Tree

- **Root Node:** the topmost node; represents the entire dataset, and the first split is made here.
- **Internal / Decision Nodes:** points where the dataset is split based on a feature.
- **Branches:** the outcome of a split (e.g., Yes/No, $\leq$ threshold or $>$ threshold).
- **Leaf / Terminal Nodes:** the final outcomes — class labels or predicted values.

How Does It Work?

- algorithm chooses the best feature and split point
 - using a criterion: Information Gain, Gain Ratio, or Gini Impurity
- recursively splits the data until a node is pure or a stopping criteria: met

Splitting: Impurity Measures

1. Gini Impurity (used in CART)

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where p_i is the probability of class i . Measures how often a randomly chosen element would be incorrectly labeled.

2. Entropy (used in ID3 (Iterative Dichotomiser 3))

$$Entropy = - \sum_{i=1}^n p_i \log_2 p_i$$

Splitting: Impurity Measures

3. Information Gain

$$IG = Entropy(Parent) - \sum \left(\frac{|Child|}{|Parent|} \right) \times Entropy(Child)$$

4. Mean Squared Error (regression trees)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

Algorithms for Building Decision Trees

Algorithm	Key Idea	Use Case
ID3	Uses entropy and information gain	Classification
C4.5	Improvement over ID3; handles continuous data	Classification
CART	Uses Gini (classification) or MSE (regression)	Classification & Regression
CHAID	Uses statistical significance tests	Multi-level categorical splits

Pruning: Avoiding Overfitting

- **Pre-pruning (Early Stopping):** stop growing the tree once a condition is met (minimum samples, max depth).
- **Post-pruning:** grow the full tree first, then remove nodes that provide little improvement.

Pros, Cons, and Applications

Advantages

- Easy to understand and visualize
- No need for feature scaling/normalization
- Works with both numerical and categorical data

Disadvantages

- Prone to overfitting
- Unstable: small data changes may produce different trees
- Biased toward features with many levels/categories

Real-world Applications

Medical diagnosis (benign vs. malignant tumors) · Credit scoring · Customer churn prediction · Fraud detection

Example: PlayGolf

14 instances, target = PlayGolf (9 Yes, 5 No). Build a tree with ID3 (entropy) and then with CART (Gini) on this same dataset.

Outlook	Temperature	Humidity	Windy	PlayGolf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

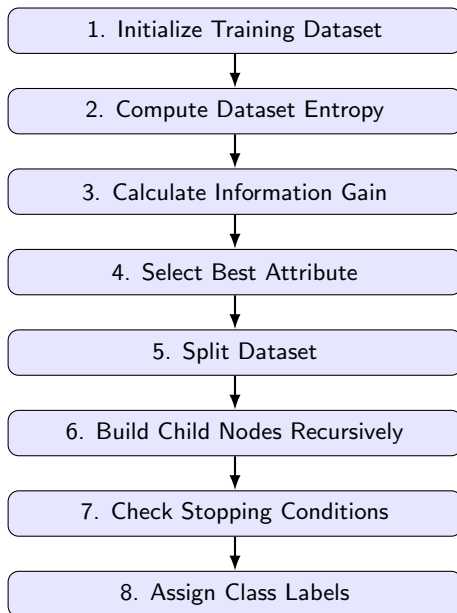
Iterative Dichotomiser 3 (ID3)

- decision tree learning algorithm: classification problems
- constructs a tree by selecting attributes that **maximize Information Gain**
 - computed using entropy
- follows a **greedy, top-down** recursive strategy to split the dataset
 - until the classification becomes pure or no attributes remain

Iterative Dichotomiser 3 (ID3)

- **Decision Tree Learning:** Represents classification logic using a tree structure
- **Entropy:** Measures impurity or randomness in the target variable
- **Information Gain:** Quantifies reduction in entropy after a split
- **Greedy Approach:** Chooses the best split at each step without backtracking
- **Categorical Attributes:** Originally designed for categorical features

ID3 Algorithm: Working Procedure



Step 1: Initialize the Dataset

Objective

Begin the decision tree construction using the complete training dataset.

- Start with the training dataset D .
- Identify the target (class) attribute.
- Treat every predictor variable as a candidate splitting attribute.
- Initially, no attribute has been selected.

Training Dataset D

Step 2: Compute Dataset Entropy

Purpose

Measure the impurity or uncertainty of the dataset.

$$Entropy(D) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where

- p_i = probability of class i
- c = number of classes

Higher entropy indicates greater disorder, while an entropy of zero indicates a perfectly pure dataset.

Step 3: Calculate Information Gain

For every candidate attribute,

- 1 Split the dataset.
- 2 Compute entropy for each subset.
- 3 Calculate weighted entropy.
- 4 Compute Information Gain.

$$Gain(D, A) = Entropy(D) - \sum_v \frac{|D_v|}{|D|} Entropy(D_v)$$

Interpretation

The larger the Information Gain, the better the attribute separates the classes.

Step 4: Select the Best Attribute

Choose the attribute having the maximum Information Gain.

$$A^* = \arg \max_A \text{Gain}(D, A)$$

Attribute	Information Gain
Weather	0.656
Humidity	0.151
Wind	0.048
Temperature	0.029

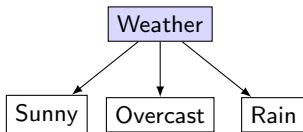
Therefore,

Weather

becomes the root node.

Step 5: Partition the Dataset

The selected attribute divides the dataset into subsets.

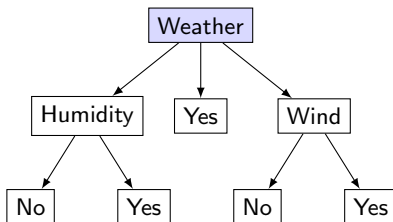


Each branch corresponds to one value of the selected attribute.

Step 6: Recursive Tree Construction

For every subset,

- 1 Remove the selected attribute.
- 2 Compute entropy again.
- 3 Calculate Information Gain.
- 4 Select the next best attribute.
- 5 Continue recursively.



Step 7: Check Stopping Conditions

The recursive process stops if any one of the following conditions is satisfied.

Stopping Conditions

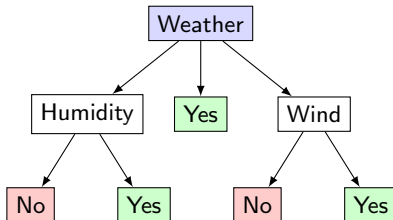
- 1 All samples belong to the same class.
- 2 No unused attributes remain.
- 3 The subset becomes empty.
- 4 Information Gain becomes zero.

In these cases, no further splitting is required.

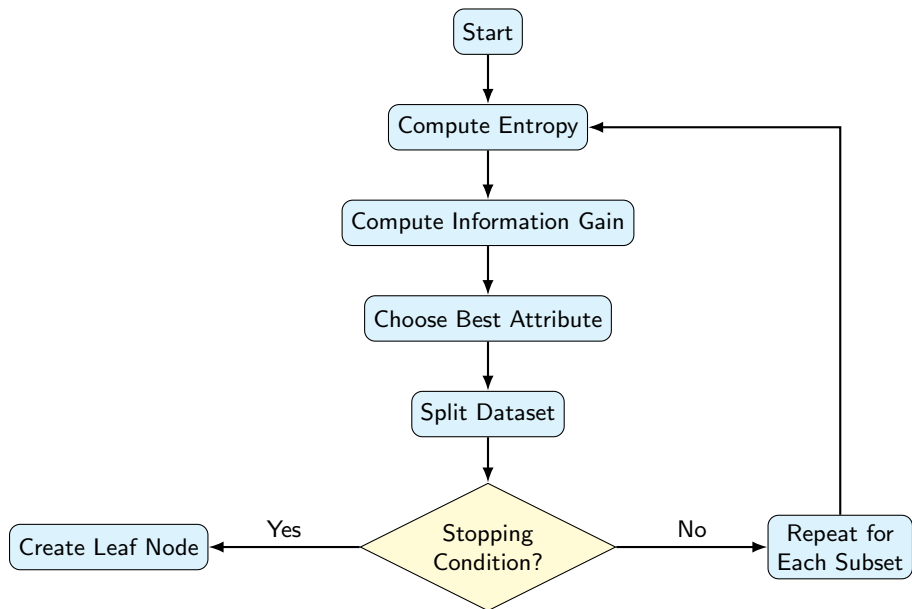
Step 8: Assign Class Labels

When recursion stops,

- Create a leaf node.
- Assign the majority class label.
- Complete the decision tree.



Complete ID3 Algorithm



ID3: Root Entropy

Total: 14 records (9 Yes, 5 No).

$$\begin{aligned} E(S) &= -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \\ &= -0.643 \log_2(0.643) - 0.357 \log_2(0.357) \approx 0.940 \end{aligned}$$

Information Gain per Attribute

For each attribute A : $IG(S, A) = E(S) - \sum_v \frac{|S_v|}{|S|} E(S_v)$

ID3: Root Entropy

Total: 14 records (9 Yes, 5 No).

$$\begin{aligned} E(S) &= -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \\ &= -0.643 \log_2(0.643) - 0.357 \log_2(0.357) \approx 0.940 \end{aligned}$$

Information Gain per Attribute

For each attribute A : $IG(S, A) = E(S) - \sum_v \frac{|S_v|}{|S|} E(S_v)$

ID3: Information Gain - Outlook

- Sunny: 5 samples (2 Yes, 3 No), $E = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \approx 0.971$
- Overcast: 4 samples (4 Yes, 0 No), $E = 0$
- Rainy: 5 samples (3 Yes, 2 No), $E \approx 0.971$

$$\begin{aligned} \text{Gain}(S, \text{Outlook}) &= 0.940 - \left(\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \right) \\ &= 0.940 - 0.693 = \mathbf{0.247} \end{aligned}$$

ID3: Information Gain - Remaining Attributes

Temperature: Hot (4, $E=1.0$), Mild (6, $E\approx 0.918$), Cool (4, $E\approx 0.811$)

$$\text{Gain}(S, \text{Temp}) = 0.940 - (0.286 + 0.393 + 0.232) = \mathbf{0.029}$$

Humidity: High (7, $E\approx 0.985$), Normal (7, $E\approx 0.591$)

$$\text{Gain}(S, \text{Humidity}) = 0.940 - \left(\frac{7}{14}(0.985) + \frac{7}{14}(0.591)\right) = 0.940 - 0.788 = \mathbf{0.152}$$

Windy: False (8, $E\approx 0.811$), True (6, $E=1.0$)

$$\text{Gain}(S, \text{Windy}) = 0.940 - \left(\frac{8}{14}(0.811) + \frac{6}{14}(1.0)\right) = 0.940 - 0.892 = \mathbf{0.048}$$

Best attribute to split on

Outlook (Gain = 0.247) - the largest gain among all four attributes

ID3: Splitting on Outlook

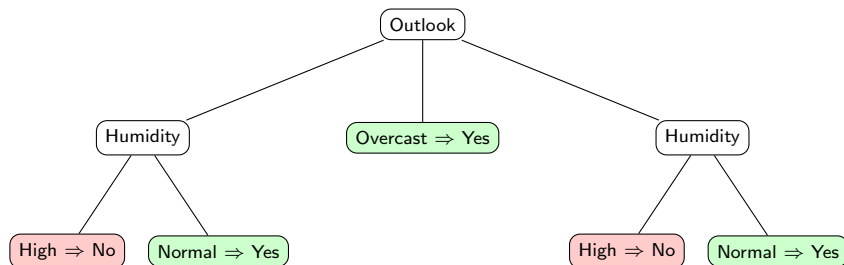
- **Outlook = Overcast:** 4 samples (4 Yes, 0 No) → **Pure node: predict Yes**
- **Outlook = Sunny:** 5 samples (2 Yes, 3 No), $E=0.971$. Split on Humidity:
 - High: 3 samples (0 Yes, 3 No), $E=0 \rightarrow$ **No**
 - Normal: 2 samples (2 Yes, 0 No), $E=0 \rightarrow$ **Yes**

(Gain = 0.971, a perfect split)

- **Outlook = Rainy:** 5 samples (3 Yes, 2 No), $E=0.971$. Split on Humidity:
 - High: 2 samples (0 Yes, 2 No), $E=0 \rightarrow$ **No**
 - Normal: 3 samples (3 Yes, 0 No), $E=0 \rightarrow$ **Yes**

(Gain = 0.971, a perfect split)

ID3: Final Decision Tree



Path	Prediction
Outlook = Overcast	Yes
Outlook = Sunny, Humidity = High	No
Outlook = Sunny, Humidity = Normal	Yes
Outlook = Rainy, Humidity = High	No
Outlook = Rainy, Humidity = Normal	Yes

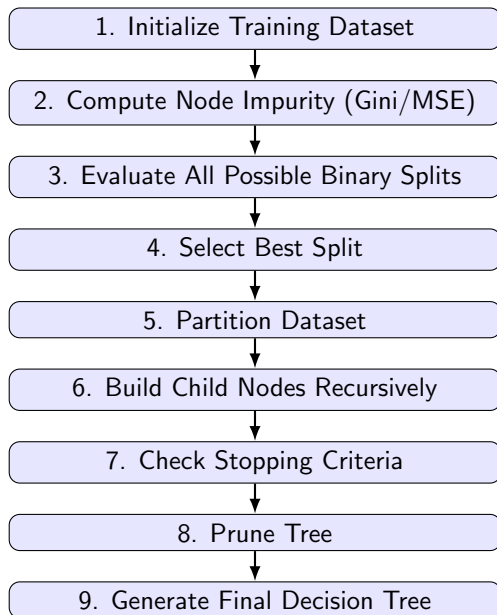
CART (Gini Index)

- used for both classification (predicting class labels) and regression (predicting continuous values)
 - Uses **Gini Index** for *classification*
 - Uses **Mean Squared Error (MSE)** for *regression*
- Unlike ID3 or C4.5, **CART produces strictly binary trees**
 - every internal node splits into **exactly two children**
- **Goal:** split data to minimize impurity

Features of CART

- Tree type: binary decision tree.
- Splitting criteria: Gini impurity (classification), MSE (regression).
- Supports both categorical and numerical attributes.
- Pruning: cost-complexity (weakest-link) pruning — grow the full tree, then prune nodes using a complexity parameter α that balances accuracy with tree size.

Working Principle of CART



Step 1: Initialize the Dataset

Objective

Start with the complete training dataset and identify the target variable.

- Load the training dataset D .
- Identify the target (class or continuous value).
- Treat every predictor variable as a candidate splitting attribute.
- Initially, the entire dataset forms the root node.

Training Dataset D

Step 2: Compute Node Impurity

Classification

Measure node impurity using the Gini Index.

$$Gini(D) = 1 - \sum_{i=1}^c p_i^2$$

where

- p_i = probability of class i
- Lower Gini indicates a purer node.

For regression problems, CART uses Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

Step 3: Evaluate All Possible Binary Splits

For every attribute,

- 1 Generate every possible binary split.
- 2 Compute Gini (or MSE) for each child node.
- 3 Calculate the weighted impurity after splitting.
- 4 Compare all possible splits.

$$Gini_{split} = \frac{|D_L|}{|D|} Gini(D_L) + \frac{|D_R|}{|D|} Gini(D_R)$$

The objective is to minimize the weighted impurity.

Step 4: Select the Best Split

Choose the split with the minimum weighted impurity.

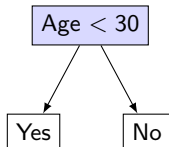
$$Split^* = \arg \min Gini_{split}$$

Candidate Split	Weighted Gini
Age < 30	0.42
Income < 50K	0.31
Student = Yes	0.27
Credit Rating	0.35

The split with the lowest Gini Index is selected.

Step 5: Partition the Dataset

The selected split divides the dataset into two subsets.

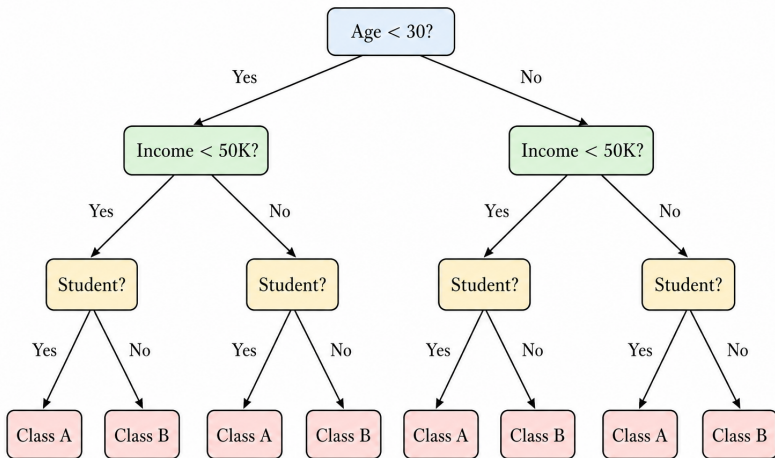


Each subset becomes a child node for further processing.

Step 6: Recursive Tree Construction

For each child node,

- 1 Compute Gini impurity.
- 2 Evaluate every possible binary split.
- 3 Select the best split.
- 4 Continue recursively until stopping criteria are met.



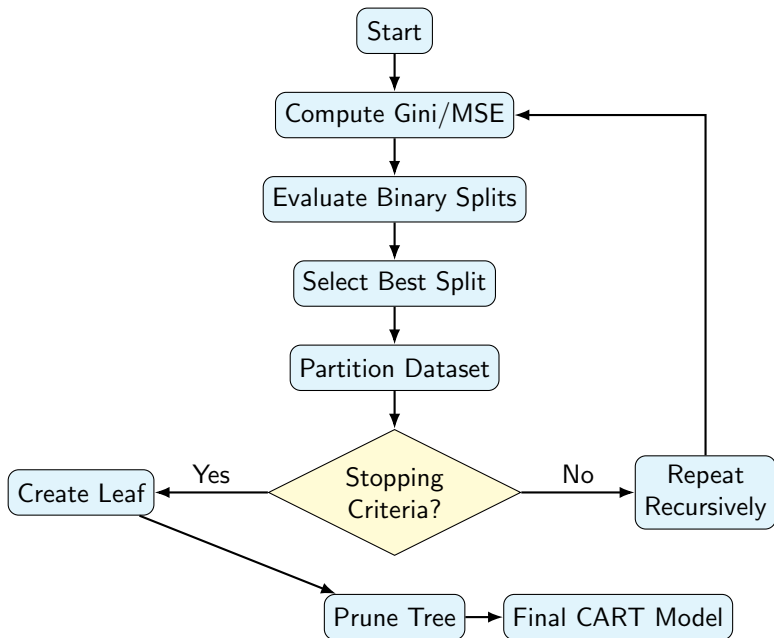
Step 7: Stopping Criteria

- All samples belong to one class.
- Maximum tree depth reached.
- Minimum number of samples reached.
- No impurity reduction.

Step 8: Pruning

- Remove weak branches.
- Reduce overfitting.
- Improve generalization.
- Simplify the tree.

Complete CART Algorithm



CART: Root Gini Index

Same PlayGolf dataset: 14 instances (9 Yes, 5 No).

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.413 - 0.128 = \mathbf{0.459}$$

Evaluate Binary Splits

CART evaluates **binary** splits only, e.g., “Outlook = Sunny? (Yes/No)”. For each candidate split we compute the weighted Gini of the two children.

CART: Candidate Binary Splits

Split: Outlook = Sunny?

- Yes (Sunny): 5 samples (2 Yes, 3 No), $Gini = 1 - (2/5)^2 - (3/5)^2 = 0.48$
- No (Not Sunny): 9 samples (7 Yes, 2 No), $Gini \approx 0.346$

$$Gini_{split} = \frac{5}{14}(0.48) + \frac{9}{14}(0.346) \approx \mathbf{0.398}$$

Split: Outlook = Overcast?

- Yes (Overcast): 4 samples (4 Yes), $Gini = 0$
- No: 10 samples (5 Yes, 5 No), $Gini = 1 - (5/10)^2 - (5/10)^2 = 0.5$

$$Gini_{split} = \frac{4}{14}(0) + \frac{10}{14}(0.5) = \mathbf{0.357}$$

CART: Candidate Binary Splits

Split: Humidity = High?

- High: 7 samples (3 Yes, 4 No), $Gini \approx 0.489$
- Not High: 7 samples (6 Yes, 1 No), $Gini \approx 0.245$

$$Gini_{split} = \frac{7}{14}(0.489) + \frac{7}{14}(0.245) = \mathbf{0.367}$$

Split: Windy = True?

- True: 6 samples (3 Yes, 3 No), $Gini = 0.5$
- False: 8 samples (6 Yes, 2 No), $Gini = 0.375$

$$Gini_{split} = \frac{6}{14}(0.5) + \frac{8}{14}(0.375) = \mathbf{0.429}$$

Best Split

Outlook = Overcast? with $Gini_{split} = 0.357$ — the lowest weighted Gini among all candidates.

CART: Candidate Binary Splits

First split: Outlook = Overcast?

- Overcast = Yes $\rightarrow$ 4 samples (4 Yes) $\rightarrow$ **Pure node: predict Yes**
- Overcast = No $\rightarrow$ 10 samples (5 Yes, 5 No) $\rightarrow$ needs further splitting

Further split on the “Overcast = No” subset (10 samples):

Try *Humidity = High*?

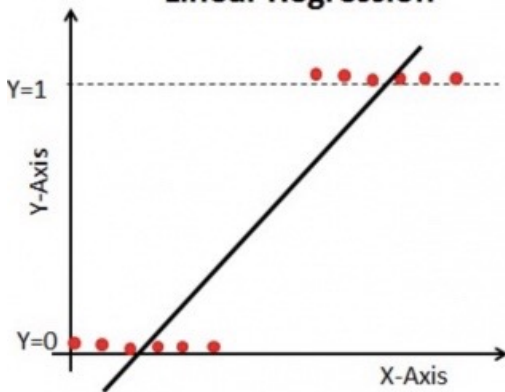
- High: 5 samples (0 Yes, 5 No), $Gini = 0$
- Not High: 5 samples (5 Yes, 0 No), $Gini = 0$

$Gini_{split} = 0 \Rightarrow$ **Perfect split!**

Result

CART converges to a clean binary tree: Outlook=Overcast $\Rightarrow$ Yes; otherwise split on Humidity, which perfectly separates the remaining 10 samples.

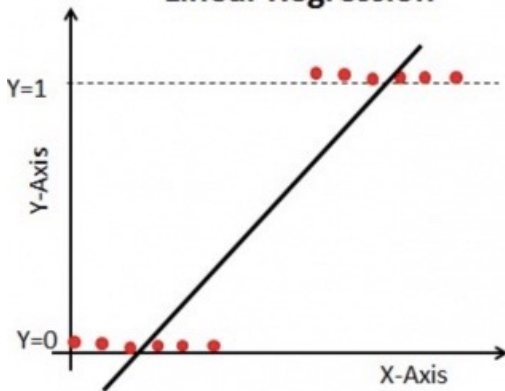
Linear Regression



$$h_{\theta}(x) = \theta^T x$$

- Threshold classifier output $h_{\theta}(x)$ at 0.5
 - If $h_{\theta}(x) \geq 0.5$, predict "y = 1"
 - If $h_{\theta}(x) < 0.5$, predict "y = 0"

Linear Regression

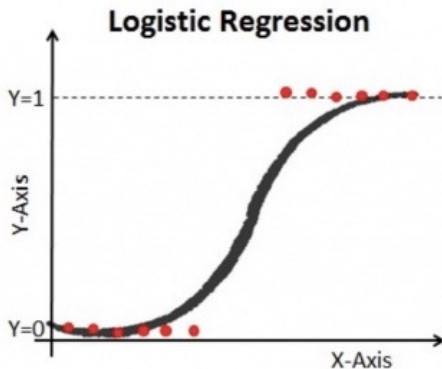


$$h_{\theta}(x) = \theta^T x$$

- Threshold classifier output $h_{\theta}(x)$ at 0.5
 - If $h_{\theta}(x) \geq 0.5$, predict "y = 1"
 - If $h_{\theta}(x) < 0.5$, predict "y = 0"
- Classification: y = 0 or y = 1
 - $h_{\theta}(x)$ can be > 1 or < 0

Classification

- Classification: $y = 0$ or $y = 1$
 - $h_{\theta}(x)$ can be > 1 or < 0
- Logistic Regression: $0 \leq h_{\theta}(x) \leq 1$



Logistic Regression

- supervised machine learning algorithm used primarily for **classification** tasks
- Unlike Linear Regression, which predicts continuous values
 - Logistic Regression estimates the **probability** that a given input belongs to a particular class
- useful for binary classification (Yes/No, 0/1, True/False)

Type	Description
Binomial Logistic Regression	Target variable has two categories (e.g., Yes/No)
Multinomial Logistic Regression	Target has 3+ unordered categories (e.g., cat/-dog/sheep)
Ordinal Logistic Regression	Target has 3+ ordered categories (e.g., Low/Med/High)

Assumptions of Logistic Regression

- **Independent Observations:** each sample is independent
- **Binary Output** (for the binary case): target variable should be 0 or 1
- **Linearity of Logit:** a linear relationship between predictors and log-odds
- **No Outliers:** outliers can distort parameter estimation
- **Sufficient Sample Size:** needed for stability and accuracy

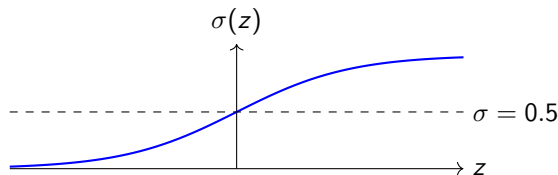
Sigmoid Function

Want $0 \leq h_{\theta}(x) \leq 1$ Modify $h_{\theta}(x) = g(\theta^T x)$

$$g(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\theta^T x}}$$

The sigmoid function transforms any real-valued number into a value between 0 and 1:

$$g(z) = \frac{1}{1 + e^{-z}}, \quad \lim_{z \rightarrow \infty} g(z) = 1, \quad \lim_{z \rightarrow -\infty} g(z) = 0$$



Asymptotic at 1, i.e., $g(z) \rightarrow 1$ as $z \rightarrow +\infty$, Asymptotic at 0, i.e., $g(z) \rightarrow 0$ as $z \rightarrow -\infty$

Decision Rule

$\sigma(z) \geq 0.5 \Rightarrow \text{Class 1;}$ otherwise Class 0.

Step 1: Linear function

$$z = \mathbf{w} \cdot \mathbf{X} + b$$

where $\mathbf{X}$ = feature vector, $\mathbf{w}$ = weights, b = bias/intercept.

Step 2: Apply the sigmoid

$$p(Y = 1|X) = g(z) = \frac{1}{1 + e^{-z}}$$

This gives the probability of belonging to Class 1.

Odds

$$\text{Odds} = \frac{p(x)}{1 - p(x)} = e^z$$

Log-Odds (Logit): the Linear Predictor

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \mathbf{w} \cdot \mathbf{X} + b$$

Interpretation of Hypothesis Output

- $h_{\theta}(x)$ = estimated probability that $y = 1$ on input x
- Example: If $\begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ x_1 \end{bmatrix}$
 - x_1 : **tumor size**, weight, email
 - $h_{\theta}(x) = 0.7$ Tell patient: 70% chance of tumor being cancerous
- $h_{\theta}(x) = p(y = 1 \mid x; \theta)$
 - probability that $y = 1$, given x , parameterized by θ
- $y = 0$ or 1

Interpretation of Hypothesis Output

- $h_{\theta}(x)$ = estimated probability that $y = 1$ on input x
- Example: If $\begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ x_1 \end{bmatrix}$
 - x_1 : **tumor size**, weight, email
 - $h_{\theta}(x) = 0.7$ Tell patient: 70% chance of tumor being cancerous
- $h_{\theta}(x) = p(y = 1 \mid x; \theta)$
 - probability that $y = 1$, given x , parameterized by θ
- $y = 0$ or 1

$$P(y = 0 \mid x; \theta) + P(y = 1 \mid x; \theta) = 1$$

$$P(y = 0 \mid x; \theta) = 1 - P(y = 1 \mid x; \theta)$$

Interpretation of Hypothesis Output

- $h_{\theta}(x)$ = estimated probability that $y = 1$ on input x
- Example: If $\begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ x_1 \end{bmatrix}$
 - x_1 : **tumor size**, weight, email
 - $h_{\theta}(x) = 0.7$ Tell patient: 70% chance of tumor being cancerous
- $h_{\theta}(x) = p(y = 1 \mid x; \theta)$
 - probability that $y = 1$, given x , parameterized by θ
- $y = 0$ or 1

$$P(y = 0 \mid x; \theta) + P(y = 1 \mid x; \theta) = 1$$

$$P(y = 0 \mid x; \theta) = 1 - P(y = 1 \mid x; \theta)$$

$$P(y \mid x) = h(x)^y (1 - h(x))^{1-y}$$

Likelihood and Log-Likelihood

Given n independent observations, the likelihood is:

$$L(\mathbf{w}, b) = \prod_{i=1}^n p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

Taking logs, the **log-likelihood** to be maximized:

$$\log L(\mathbf{w}, b) = \sum_{i=1}^n \left[y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i)) \right]$$

No closed-form solution exists; parameters are found via numerical optimization (Maximum Likelihood Estimation, MLE).

Gradient for Optimization

Gradient (Gradient Ascent on the log-likelihood J):

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^n (y_i - p(x_i)) x_{ij}$$

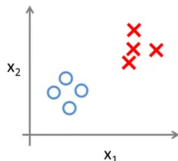
Weight update rule:

$$w_j := w_j + \alpha \cdot \frac{\partial J}{\partial w_j} \quad (\text{simultaneously update all } w_j)$$

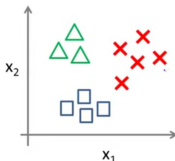
where α is the learning rate. This is the basis of Iteratively Reweighted Least Squares / gradient-based solvers used in practice (e.g., Newton–Raphson, L-BFGS).

Multiclass Classification

Binary classification:



Multi-class classification:



Andrew Ng

- Email folder: Work, Personal, Family, Friends, Spam
- $y \in \{0, 1, 2, 3, 4\}$
- Idea: transform multi-class problems into multiple binary classification problems
- Two approaches:
 - One versus All / One vs Rest
 - One versus One

Softmax: Multiclass Logistic Regression

For K classes, the **softmax function** generalizes the sigmoid:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Final probability for class c :

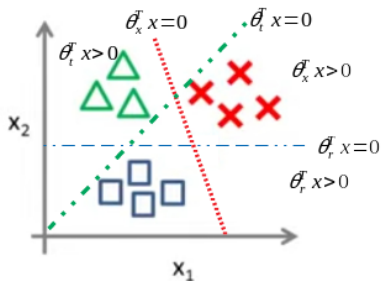
$$P(y = c \mid x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{k=1}^K e^{w_k \cdot x + b_k}}$$

Used for **Multinomial** Logistic Regression when the target has 3+ unordered categories.

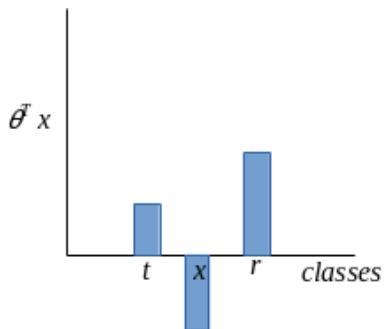
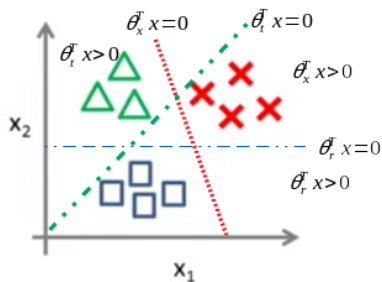
Softmax Regression

- $K = \text{classes} \in \{1, \dots, k, \dots, K\}$
- $x^i \in \mathbb{R}^n$
- $y = \{0, 1\}^n$
- $\theta_k \in \mathbb{R}^n$

Assume three classes (r , t , and x)



Softmax Regression

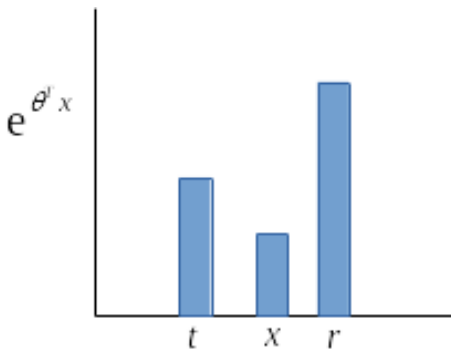


Raw scores $\theta_k^T x$ per class (can be negative)

Softmax Regression

The softmax function converts raw scores into a valid probability distribution over the K classes:

$$\tilde{p}(y = k) = \frac{e^{\theta_k^T x}}{\sum_{k' \in K} e^{\theta_{k'}^T x}}$$



All bars are non-negative and sum to 1 — a proper probability distribution over $\{t, x, r\}$.

Logistic vs. Linear Regression

Aspect	Linear Regression	Logistic Regression
Output	Continuous value	Probability (0 to 1)
Problem Type	Regression	Classification
Estimation	Least Squares	Maximum Likelihood
Model Shape	Best-fit line	Sigmoid (S-shaped) curve
Output Range	$(-\infty, +\infty)$	$(0, 1)$
Assumption	Linearity in X, Y	Linearity in X , log-odds

Term	Description
Odds	$\frac{P}{1-P}$
Logit	$\log\left(\frac{P}{1-P}\right)$
Coefficients	Weights showing effect of input features
Intercept	Constant term in the logit equation
MLE	Maximum Likelihood Estimation for parameter learning

Evaluation Metrics

Metric	Formula	Use Case
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	General performance
Precision	$\frac{TP}{TP+FP}$	Importance of positive class
Recall	$\frac{TP}{TP+FN}$	Catching all positive cases
F1-Score	$2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$	Trade-off between P and R
AUC-ROC	Area under ROC Curve	Probabilistic ranking quality

Example

Dataset (single feature x , binary target y):

x	y
2	0
4	0
6	1
8	1

Let the (fitted) parameters be $\theta_0 = -4$, $\theta_1 = 1$.

Prediction for $x = 6$:

$$z = \theta_0 + \theta_1 x = -4 + 1(6) = 2, \quad h_{\theta}(x) = \frac{1}{1 + e^{-2}} \approx \frac{1}{1.1353} \approx \mathbf{0.88}$$

Interpretation

An 88% probability the class is 1 $\Rightarrow$ predict **1**.

Worked Numerical Example (contd.)

Prediction for $x = 2$:

$$z = \theta_0 + \theta_1 x = -4 + 1(2) = -2$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + e^2} \approx \frac{1}{8.389} \approx \mathbf{0.119}$$

Interpretation

About 11.9% chance the class is 1 $\Rightarrow$ predict **0**. Notice the model's decision boundary is exactly $z = 0 \Rightarrow x = 4$, consistent with the observed data ($x = 2, 4 \rightarrow y = 0$; $x = 6, 8 \rightarrow y = 1$).

Applications, Advantages, Disadvantages

Applications

Email spam detection · Disease prediction (e.g., diabetes) · Credit risk scoring · Marketing campaign success prediction

Advantages

- Simple, fast to implement
- Probabilistic interpretation of outcomes
- Works well with linearly separable data

Disadvantages

- Assumes a linear decision boundary
- May underperform with non-linear/intricate feature-target relationships

Check Your Understanding: Logistic Regression

Analytical Question

In the worked example, the fitted decision boundary is $z = 0 \Leftrightarrow x = 4$. (a) Using $\theta_0 = -4, \theta_1 = 1$, compute $h_{\theta}(x)$ for $x = 4$ exactly, and explain why this is the natural decision boundary. (b) A colleague suggests interpreting $\theta_1 = 1$ as “one unit of x increases the probability of $y = 1$ by 1.” Explain why this interpretation is incorrect, and state what θ_1 actually represents (in terms of log-odds and odds ratio e^{θ_1}).

Permutation Feature Importance

Objective

To quantify the importance of each feature by measuring the change in model performance when the feature's values are randomly shuffled, breaking the relationship between that feature and the target.

- 1: Train the model on training data.
- 2: Evaluate a baseline performance metric: $M_{base} = \text{metric}(y, \hat{y})$
- 3: **for** each feature X_i **do**
- 4: Shuffle its values in the validation/test set.
- 5: Recalculate predictions and evaluate the performance metric M_i .
- 6: Compute importance score: $I_i = M_i - M_{base}$
- 7: **end for**

Permutation Importance

- **Model-agnostic:** works with any type of model.
- **Captures feature interactions.**
- **Limitation:** sensitive to correlated features.

Pros

- Simple, intuitive
- Supports any model

Cons

- Computationally expensive
- May underestimate importance of correlated features

Worked Numerical Example: Permutation Importance

A Random Forest classifier predicts *Churn* (Yes/No). Baseline test-set error (misclassification rate): $M_{base} = 0.10$.

Predictor	M_i (after shuffle)	$I_i = M_i - M_{base}$	Rank
Contract Type (categorical)	0.24	0.14	1
Monthly Charges (numeric)	0.17	0.07	2
Tenure (numeric)	0.15	0.05	3
Payment Method (categorical)	0.11	0.01	4

Interpretation

Contract Type is the most influential predictor: permuting it raises the error rate the most. *Payment Method* contributes little.

Partial Dependence Plots (PDP): Objective

Objective

To visualize the marginal effect of one or two features on the predicted outcome by averaging out the effects of all other features.

Mathematical Formulation

Let $f(x)$ be the trained prediction function. The partial dependence of feature subset X_s is:

$$PD(X_s) = \mathbb{E}_{X_c} [f(X_s, X_c)]$$

where X_s is the feature(s) of interest, X_c is the complement set of all other features, and the expectation is taken over the joint distribution of X_c .

PDP: Procedure and Key Points

Procedure

- 1 Select a feature X_s and a range of values for it.
- 2 For each value, substitute it in all rows of the dataset and make predictions.
- 3 Average the predictions to estimate the marginal effect at that value.

Key Points

- Visual interpretation: 1D or 2D plots.
- **Assumes feature independence** — can be misleading if features are correlated.
- Often used with tree-based models (Random Forests, Gradient Boosted Trees).

Partial Dependence: Numerical Illustration

Suppose a fitted model predicts *Default Probability* from *Income*, averaging over all other features:

Income (\$1000s)	20	40	60	80	100
Avg. Predicted $P(\text{Default})$	0.42	0.31	0.22	0.17	0.15

Interpretation

A monotone decreasing, concave relationship: default probability drops sharply between \$20k–\$60k income, then **plateaus** — diminishing marginal effect of income beyond \$60k.

Comparison: Permutation Importance vs. PDP

Aspect	Permutation Importance	Partial Dependence Plot
Type	Quantitative metric	Visual interpretation
Output	Importance score	Plot of average model output
Captures Interactions	Yes	Yes (in 2D plots)
Feature Correlation Issue	Yes (underestimates importance)	Yes (can be misleading)
Model Compatibility	Any (model-agnostic)	Mostly tree-based models
Usefulness	Feature ranking	Understanding effect on prediction
Cost	Medium to High	Medium

Table 4.1: Comparison of Permutation Importance and PDP

Check Your Understanding: Variable Importance

Analytical Question

Two predictors, *Age* and *Years of Employment*, are highly correlated ($r = 0.85$). A permutation importance analysis ranks *Age* as the top predictor and *Years of Employment* as unimportant. (a) Explain a mechanism by which this ranking could be misleading due to correlation, referencing the “Feature Correlation Issue” row of Table 4.1. (b) Would a Partial Dependence Plot of *Years of Employment* resolve this ambiguity on its own? Justify your answer using the “assumes feature independence” limitation.

K-Means Clustering

Definition

K-Means is an **unsupervised, iterative** clustering technique that partitions a dataset into k predefined distinct clusters. A cluster is a collection of data points exhibiting certain similarities.

Cluster Properties

- Each data point belongs to a cluster whose mean (centroid) is nearest to it.
- Data points within the same cluster exhibit high similarity.
- Data points from different clusters show high dissimilarity.

K-Means Clustering Algorithm

- 1: **Step 1:** Choose the number of clusters k .
- 2: **Step 2:** Initialize cluster centers — randomly select k data points, ideally as far apart as possible.
- 3: **Step 3:** Assign data points — compute the distance (commonly Euclidean) from each point to each cluster center.
- 4: **Step 4:** Assign each point to the cluster with the nearest centroid.
- 5: **Step 5:** Recompute centroids — for each cluster, take the mean of all assigned points.
- 6: **Step 6:** Repeat Steps 3–5 until centroids/assignments stop changing, or a maximum number of iterations is reached.

K-Means: Advantages and Disadvantages

Advantages

- **Efficiency:** time complexity $O(nkt)$, where n = points, k = clusters, t = iterations.
- **Fast convergence**, though it may reach a local optimum.
- **Extensibility:** Simulated Annealing or Genetic Algorithms can help escape local optima.

Disadvantages

- Requires k to be specified in advance.
- Sensitive to noisy data and outliers.
- Poor performance on non-convex clusters or clusters of varying density/size.

Worked Example 1: K-Means, $K = 2$

Given: $K = 2$; initial centroids $C_1 = (185, 72)$, $C_2 = (170, 56)$.

Point	Coordinates
P_1	(185, 72)
P_2	(170, 56)
P_3	(168, 60)
P_4	(179, 68)
P_5	(182, 72)
P_6	(188, 77)

Distance formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Worked Example 1: Iteration 1

Point	$d(P, C_1)$	$d(P, C_2)$	Cluster
P_1 (185,72)	0	21.93	C_1
P_2 (170,56)	21.93	0	C_2
P_3 (168,60)	20.80	4.47	C_2
P_4 (179,68)	7.21	15.00	C_1
P_5 (182,72)	3.00	20.00	C_1
P_6 (188,77)	5.83	27.66	C_1

Clusters after Iteration 1: $C_1 = \{P_1, P_4, P_5, P_6\}$, $C_2 = \{P_2, P_3\}$

New centroids:

$$C'_1 = \left(\frac{185+179+182+188}{4}, \frac{72+68+72+77}{4} \right) = (183.5, 72.25)$$

$$C'_2 = \left(\frac{170+168}{2}, \frac{56+60}{2} \right) = (169, 58)$$

Worked Example 1: Iteration 2 and Convergence

Point	$d(P, C'_1)$	$d(P, C'_2)$	Cluster
P_1 (185,72)	1.52	21.26	C_1
P_2 (170,56)	21.13	2.24	C_2
P_3 (168,60)	19.71	2.24	C_2
P_4 (179,68)	6.20	14.14	C_1
P_5 (182,72)	1.52	19.10	C_1
P_6 (188,77)	6.55	26.87	C_1

Clusters after Iteration 2: $C_1 = \{P_1, P_4, P_5, P_6\}$, $C_2 = \{P_2, P_3\}$ — **unchanged** from Iteration 1.

Conclusion

The algorithm has **converged**. Final clusters:

$C_1 = \{(185, 72), (179, 68), (182, 72), (188, 77)\}$; $C_2 = \{(170, 56), (168, 60)\}$.

Worked Example 2: K-Means, $K = 3$ (8 points, 2 iterations)

Data points:

Point		Coordinates		
A_1	(2,10)	A_2 (2,5)	A_3 (8,4)	A_4 (5,8)
A_5	(7,5)	A_6 (6,4)	A_7 (1,2)	A_8 (4,9)

Initial centers: $C_1=A_1=(2, 10)$, $C_2=A_4=(5, 8)$, $C_3=A_7=(1, 2)$

Worked Example 2: Iteration 1

Point	$d(C_1)$	$d(C_2)$	$d(C_3)$	Cluster
A_1 (2,10)	0.00	3.61	8.06	C_1
A_2 (2,5)	5.00	3.61	3.16	C_3
A_3 (8,4)	8.48	5.00	7.28	C_2
A_4 (5,8)	3.61	0.00	6.71	C_2
A_5 (7,5)	7.62	3.61	6.08	C_2
A_6 (6,4)	7.21	4.47	5.39	C_2
A_7 (1,2)	8.06	6.71	0.00	C_3
A_8 (4,9)	2.24	1.41	7.28	C_2

Clusters: $C_1 = \{A_1\}$, $C_2 = \{A_3, A_4, A_5, A_6, A_8\}$, $C_3 = \{A_2, A_7\}$

New centroids: $C_1 = (2, 10)$; $C_2 = \left(\frac{8+5+7+6+4}{5}, \frac{4+8+5+4+9}{5}\right) = (6, 6)$;

$C_3 = \left(\frac{2+1}{2}, \frac{5+2}{2}\right) = (1.5, 3.5)$

Worked Example 2: Iteration 2

Point	$d(C_1:2, 10)$	$d(C_2:6, 6)$	$d(C_3:1.5, 3.5)$	Cluster
A_1 (2,10)	0.00	5.00	6.54	C_1
A_2 (2,5)	5.00	4.12	1.80	C_3
A_3 (8,4)	8.48	2.83	6.55	C_2
A_4 (5,8)	3.61	2.24	5.70	C_2
A_5 (7,5)	7.62	1.41	5.70	C_2
A_6 (6,4)	7.21	2.00	4.53	C_2
A_7 (1,2)	8.06	5.00	1.58	C_3
A_8 (4,9)	2.24	3.61	6.40	C_1

Clusters after Iteration 2: $C_1 = \{A_1, A_8\}$, $C_2 = \{A_3, A_4, A_5, A_6\}$, $C_3 = \{A_2, A_7\}$ — note A_8 moved from C_2 to C_1 .

Worked Example 2: Final Answer

New centroids after Iteration 2:

$$C_1 = \left(\frac{2+4}{2}, \frac{10+9}{2} \right) = (3, 9.5)$$

$$C_2 = \left(\frac{8+5+7+6}{4}, \frac{4+8+5+4}{4} \right) = (6.5, 5.25)$$

$$C_3 = \left(\frac{2+1}{2}, \frac{5+2}{2} \right) = (1.5, 3.5)$$

Final Cluster Centers (after 2 iterations)

Cluster 1: (3, 9.5) Cluster 2: (6.5, 5.25) Cluster 3: (1.5, 3.5)

4.4.1 The Problem: K-Means and Mixed-type Data

Handling mixed-type data (numerical + categorical variables) requires specialized approaches, since standard K-means and Euclidean distance are **incompatible** with categorical features.

One-Hot Encoding + Euclidean Distance

Approach: convert categorical variables to binary columns.

Issues:

- High dimensionality and sparsity distort distance.
- Centroids become non-interpretable (fractional values).
- Suitable only for low-cardinality categorical features.

Gower Distance

The ideal unified metric for mixed data: numerical variables use Manhattan distance normalized by range; categorical variables use Dice similarity (1 if same, 0 if different).

$$d_{\text{Gower}}(x_i, x_j) = \frac{\sum_{k=1}^p w_k \delta_{ijk}}{\sum_{k=1}^p w_k}$$

where δ_{ijk} is the feature-specific distance and w_k is an optional weight.

- **Advantages:** natural mixed-type handling; normalized $[0, 1]$ distances.
- **Limitation:** $O(n^2)$ computational complexity.

Other Metrics

Hamming Distance: mismatch count for categorical features. **Weighted Combinations:** custom blends (e.g., Euclidean + Hamming).

Worked Numerical Example: Gower's Distance

Two patients, variables: *Age* (numeric, data range = 40), *Blood Type* (categorical), *Smoker* (categorical: Yes/No).

Patient	Age	Blood Type	Smoker
i	30	A	Yes
i'	50	B	Yes

$$s_{\text{Age}} = 1 - \frac{|30 - 50|}{40} = 0.5, \quad s_{\text{BloodType}} = 0 \text{ (A} \neq \text{B)}, \quad s_{\text{Smoker}} = 1 \text{ (Yes = Yes)}$$

$$S_G = \frac{0.5 + 0 + 1}{3} = 0.5 \quad \Rightarrow \quad d_G = 1 - 0.5 = 0.5$$

4.4.2 K-Prototypes

A hybrid of K-Means (numerical part) and K-Modes (categorical part):

$$d(x_i, x_j) = \sum_{k \in \text{num}} (x_{ik} - x_{jk})^2 + \gamma \sum_{k \in \text{cat}} \mathbb{I}(x_{ik} \neq x_{jk})$$

where γ balances the contribution of numeric vs. categorical features.

- Numeric part of centroid: cluster **mean** (as in K-means).
- Categorical part of centroid: cluster **mode** (most frequent category, as in K-modes).

K-Prototypes: Implementation

```
from kmodes.kprototypes import KPrototypes
# Preprocess: Scale numericals, encode categorical as integers
kp = KPrototypes(n_clusters=3, init='Cao')
clusters = kp.fit_predict(data, categorical=[0, 2])
```

Choosing γ

A common rule of thumb: set γ to the average standard deviation of the numeric variables, so categorical mismatches are penalized on a comparable scale to numeric squared differences.

K-Medoids with Gower Distance

The PAM (Partitioning Around Medoids) algorithm uses actual data points as centroids.

- Robust to noise.
- Compatible with any precomputed distance matrix.

```
import gower
from sklearn_extra.cluster import KMedoids
dist_matrix = gower.gower_matrix(data)
kmed = KMedoids(n_clusters=3, metric="precomputed")
labels = kmed.fit_predict(dist_matrix)
```

Factor Analysis of Mixed Data (FAMD) is a dimensionality-reduction technique for mixed data:

- Projects data into a continuous latent space.
- Preserves variance in both numeric and categorical feature types.

```
from prince import FAMD
from sklearn.cluster import KMeans

famd = FAMD(n_components=5).fit(data)
kmeans = KMeans(n_clusters=3).fit(famd.row_coordinates_)
```

Practical Considerations

4.4.3 Preprocessing

- Scale numerical features (e.g., StandardScaler).
- Encode categoricals as integers or one-hot.

Balancing Feature Types & Validation

- Use the γ parameter in K-Prototypes; apply weights in Gower distance.
- **Cluster validation:** Silhouette Score with Gower distance; inspect cluster characteristics (categorical modes).

Limitations

K-Prototypes assumes spherical clusters; Gower distance scales poorly for large datasets; there is no universal solution — experimentation is required.

Method Comparison for Mixed-Type Clustering

Method	Distance Metric	Algorithm	Scalability
K-Prototypes	Hybrid (Euclidean + Dissimilarity)	Custom	High
Gower + K-Medoids	Gower	PAM	Low
One-Hot + K-Means	Euclidean	K-Means	Medium
FAMD + K-Means	Euclidean (latent space)	K-Means	Medium

Table 4.2: Summary of Clustering Methods for Mixed-Type Data

Recommendations

K-Prototypes: best for efficiency and implementation simplicity. **Gower + K-Medoids:** preferred for accuracy in small-to-medium datasets. Always validate with domain knowledge and multiple metrics.

Case Study: Customer Segmentation (Mixed-type)

Scenario

A retailer segments customers using: *Annual Spend* (numeric), *Age* (numeric), *Preferred Channel* (categorical: Online/In-store/Both), *Loyalty Tier* (ordinal: Bronze/Silver/Gold).

Applying K-Prototypes with $K = 3$ might reveal:

- **Cluster 1:** young, high online spend, Bronze/Silver tier — “Digital Bargain Shoppers.”
- **Cluster 2:** older, moderate in-store spend, Gold tier — “Loyal In-store Regulars.”
- **Cluster 3:** middle-aged, high spend across both channels, Gold tier — “High-Value Omnichannel Customers.”

Each centroid combines a **numeric mean** (spend, age) and a **categorical mode** (channel, tier) — directly actionable for targeted marketing.

Check Your Understanding: Clustering

Analytical Question

A retailer clusters customers using one-hot encoded *Preferred Channel* (3 categories) together with raw *Annual Spend* (range \$0–\$50,000) using standard Euclidean K-means. (a) Explain why this approach is likely to produce clusters driven almost entirely by *Annual Spend*, referencing the issues with one-hot encoding discussed above. (b) Propose two concrete remedies — one using Gower's distance, one using K-Prototypes with an appropriately tuned γ .

Connecting the Threads

